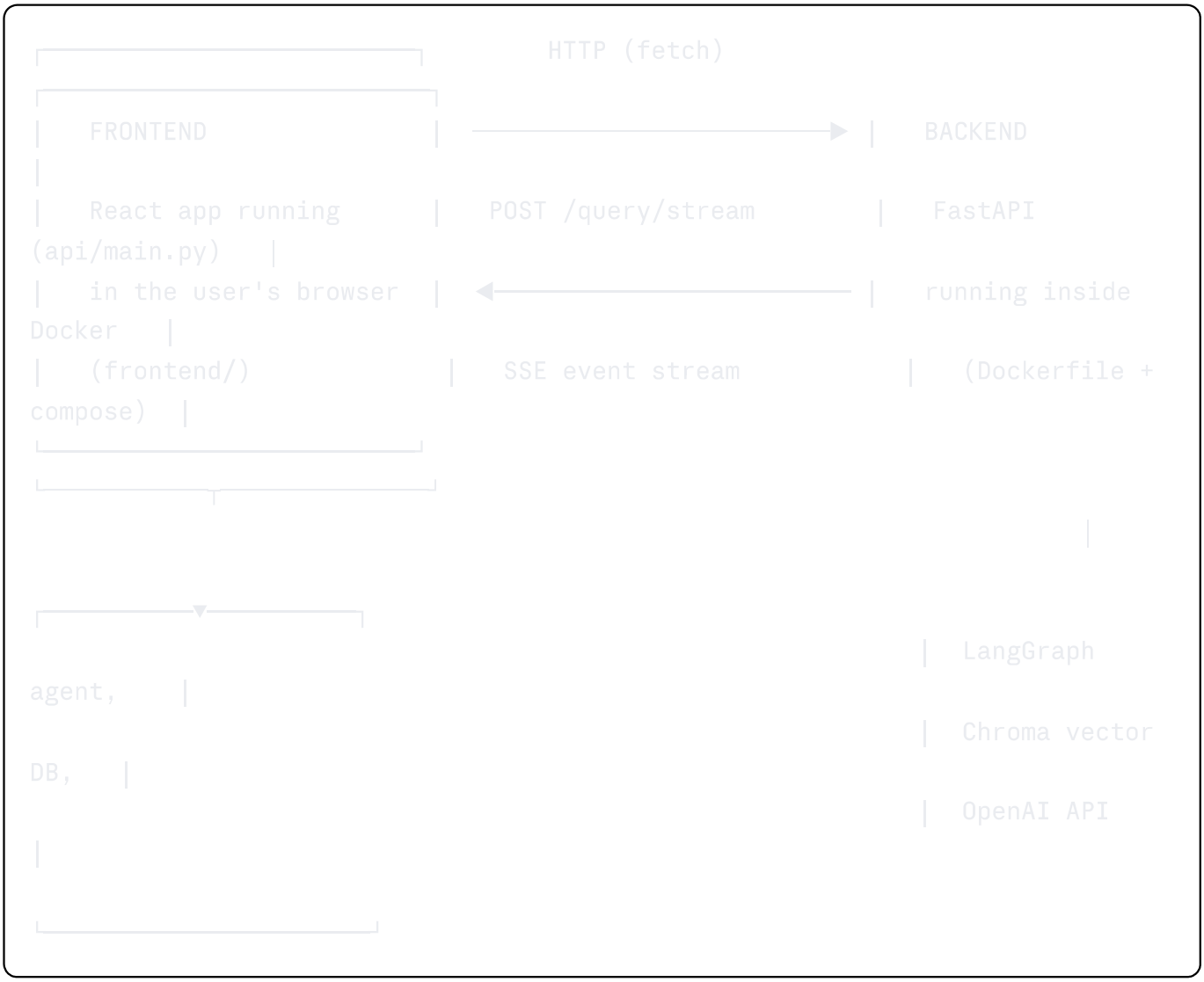
Frontend & Docker, from first principles — via your `corrective-rag-agent` repo

You know backend. This guide leans on that: every frontend and Docker concept gets tied back to a backend analogy, and every explanation is grounded in the actual files sitting in your repo, not generic tutorial code.

PART 0 — The big picture

Your project is really **two independent programs** that happen to talk to each other over HTTP:



Two things fall out of this that surprise people coming from backend-only work:

1. **The frontend never runs "on your server."** It's a bundle of HTML/CSS/JS files that Vercel (or Vite's dev server) hands to the *browser*, and the browser executes it. There is no Python, no process, no container for the frontend in production — just static files. That's why there's no `frontend` service in `docker-compose.yml`.
2. **Docker in this repo exists only to package the backend.** It solves "how do I ship a Python process with the right interpreter, packages, and OS libraries to a server" — a problem the frontend, being static files, simply doesn't have.

Keep that split in mind — it explains almost every structural decision below.

PART 1 — Frontend, from first principles

1.1 What "frontend" even means

Backend code runs on a machine you control and produces *data* (JSON, HTML, whatever). Frontend code runs on the **user's** machine, inside their browser, and its job is to turn data into pixels the user can see and click.

Every browser frontend, no matter how fancy, is built from three languages doing three separate jobs:

Language	Job	Rough analogy
HTML	structure / what elements exist	a database schema — defines *what things are*
CSS	appearance	formatting rules applied on top
JavaScript	behavior — reacts to clicks, fetches data, updates the page	your route handlers — the actual logic

Your app is React + Tailwind + Vite, which is JavaScript generating HTML, and CSS generated from utility class names. Nothing here is a new language — it's tooling layered on top of those same three things.

1.2 Why not just write plain HTML/CSS/JS?

Your chat UI needs to: append new messages, stream reasoning steps in live, disable the input while streaming, swap layouts on mobile vs desktop, re-render citation lists, etc. — the page's content is constantly changing based on data. Manually writing code that finds the right `<div>` and patches it every time something changes gets unmanageable fast (and is a classic source of bugs: forgetting to update one place the state is shown).

React's pitch: you describe *what the UI should look like given the current state*, as a plain JavaScript function. React figures out what changed and patches only the DOM nodes that need patching. You never manually call `document.getElementById(...).innerHTML = ...` anywhere in your codebase — check ChatPanel.jsx, ReasoningTrace.jsx, anywhere — that's the tell that this is a proper React app.

1.3 React fundamentals

Components are just functions

```jsx
```

```
// frontend/src/components/RetryBadge.jsx
export default function RetryBadge({ retriesUsed }) {
  const hasRetries = retriesUsed > 0
  ...
  return (
    <span className={...}>
      <Icon className="h-3 w-3" strokeWidth={2.5} />
      {label}
    </span>
  )
}
```

`RetryBadge` is a JavaScript function. It takes one argument — `{ retriesUsed }`, destructured out of a single object — and *returns a description of UI*. That's a **component**. Nothing more mystical than that.

JSX is not HTML

The `<span className={...}>...</span>` syntax above looks like HTML but it is not — it's **JSX**, a syntax extension that Vite/Babel compiles into plain JavaScript function calls (`React.createElement('span', {...}, ...)`) before it ever reaches the browser. This is why the browser never sees `<span>` as literal markup — by the time it runs, it's already ordinary JS building a tree of objects (React's "virtual DOM").

Two JSX quirks worth knowing because you'll see them constantly in this repo:

- `className` instead of `class` (`class` is a reserved word in JS).
- `{expression}` embeds a JS expression's *value* into the markup — e.g. `{label}` in RetryBadge, or the ternary in `{isStreaming ? <span .../> : <ArrowUp .../>}` in `ChatPanel.jsx`.

Props: read-only inputs, passed down

`retriesUsed` is a **prop** — an argument passed *into* a component by its parent. Trace it: `ChatPanel.jsx` renders `<RetryBadge retriesUsed={message.retries_used} />` for each message. Props flow strictly **one direction, parent → child**, exactly like a function call — a component can't reach up and modify its parent's data. If a child needs to *cause* a change in the parent, the parent hands it a callback function as a prop instead (see `onSubmit` below).

State: data a component remembers, that triggers re-renders when it changes

jsx

```
// frontend/src/components/ChatPanel.jsx
const [input, setInput] = useState('')
```

`useState('')` gives you two things: the current value (`input`, starts as `''`) and a setter function (`setInput`). Calling `setInput(newValue)` does two things: (1) updates the stored

value, (2) tells React "re-run this component function, something it depends on changed." That re-run is how the text box visibly updates as you type — every keystroke calls `setInput(event.target.value)` in the `onChange` handler, which triggers a re-render with the new value plugged into `value={input}`.

This "controlled input" pattern — the DOM input's displayed value is always driven *from* React state, never read *out of* the DOM directly — is the default way React handles forms. It feels backwards coming from `request.form['field']` style backend thinking, but the payoff is that `input` is always the single source of truth, and other code (like the disabled-submit-button check `disabled={isStreaming || !input.trim()}`) can just read it.

The render cycle, in one sentence

State or props change → component function re-executes → React builds a new virtual-DOM description → React diffs it against the previous one → React applies only the minimal real DOM changes needed. You write the "describe the end state" function; React handles the "figure out the diff and patch efficiently" part.

Hooks: functions that plug into React's machinery

Any function starting with `use` is a **hook**. Hooks can only be called at the top level of a component (or another hook) — never inside `if`s or loops — because React tracks hook state by call *order* across renders. Your codebase uses four:

- **`useState`** — as above. Used in `ChatPanel` (the input box), `InfoTooltip` (open/closed), `useChat` (messages, trace, streaming flag, error), `useMediaQuery` (matches boolean).
- **`useEffect`** — run a side effect after render, optionally re-running when specific values change, optionally cleaning up before the next run. Look at `useMediaQuery.js`:

```js
useEffect(() => {
  const mediaQueryList = window.matchMedia(query)
  const handleChange = (event) => setMatches(event.matches)
  setMatches(mediaQueryList.matches)
  mediaQueryList.addEventListener('change', handleChange)
  return () => mediaQueryList.removeEventListener('change', handleChange)
}, [query])
```

The function body runs after render; the returned function is the **cleanup**, which React calls before the effect re-runs (if `query` changes) or when the component unmounts. This acquire/release pairing is exactly the pattern you already know from context managers or connection-pool checkouts on the backend — acquire a resource (a `matchMedia` listener), guarantee it gets released. `InfoTooltip.jsx` does the identical thing for a "click outside this popover to close it" document-level listener.

The `[query]` **dependency array** tells React when to re-run the effect — only when `query` changes between renders, not on every render.

- `useRef` — a mutable box (`{ current: ... }`) that survives across renders *without* causing a re-render when you mutate it. `InfoTooltip.jsx` uses `containerRef` to get a handle on the actual DOM node, so the click-outside check (`!containerRef.current.contains(event.target)`) can ask "was the click physically outside this element" — something you can only answer by inspecting the real DOM, which is why `useRef` exists alongside React's otherwise-declarative model.
- `useMemo` — recompute a derived value only when its dependencies change, not on every render. `App.jsx`:

```
js

  const latestDisclaimer = useMemo(
    () => messages[messages.length - 1]?.disclaimer,
    [messages],
  )
```

Cheap here, but the pattern matters more in `useRepeatFlags` inside `ReasoningTrace.jsx`, where recomputing on every render would mean re-walking the whole event list on every keystroke elsewhere in the app — `useMemo` pins that work to only re-run when `events` actually changes.

Custom hooks: your own reusable stateful logic

A "custom hook" is just a function, named `useSomething`, that calls other hooks internally and returns whatever a component needs. `useChat.js` is the most important one in this app — it *is* the app's brain. Full walkthrough:

```
js
```

```
// frontend/src/hooks/useChat.js
export function useChat() {
  const [messages, setMessages] = useState([])
  const [activeTrace, setActiveTrace] = useState([])
  const [isTraceOpen, setIsTraceOpen] = useState(false)
  const [isStreaming, setIsStreaming] = useState(false)
  const [error, setError] = useState(null)

  const submitQuestion = useCallback(async (question) => {
    const trimmed = question.trim()
    if (!trimmed || isStreaming) return

    setError(null)
    setActiveTrace([])
    setIsTraceOpen(true)
    setIsStreaming(true)

    const trace = []   // <-- a plain local array, not state

    await streamQuery(trimmed, {
      onNodeUpdate: (event) => {
        trace.push(event)
        setActiveTrace([...trace])
      },
      onDone: (payload) => {
        setMessages((prev) => [...prev, { id: nextMessageId++, question: trimme
        setIsStreaming(false)
        setIsTraceOpen(false)
      },
      onError: (err) => {
        setError(err.message ?? 'Something went wrong talking to the agent.')
        setIsStreaming(false)
      },
    })
  }, [isStreaming])

  return { messages, activeTrace, isTraceOpen, setIsTraceOpen, isStreaming, err
}
```

Two subtle-but-important decisions here, both called out in the code's own comments:

1. **Why `trace` is a plain local array, not `useState`.** `setActiveTrace` is asynchronous and batched — if `onDone` read from `activeTrace` (the state variable) it could see a *stale* snapshot from before the last update finished applying, a classic "stale closure" bug in React. Keeping a plain mutable `trace` array in the closure sidesteps that entirely:

`onDone` reads the array it's been pushing into directly, no race with React's scheduler. `setActiveTrace([...trace])` still updates state separately, purely so the UI re-renders with each new step.

2. `nextMessageId` **lives outside the hook, at module scope.** It's declared once when the file loads (`let nextMessageId = 1`) and increments forever — it survives across every call to `useChat()`, which a variable declared *inside* the hook (reset via `useState`) wouldn't. This gives every message a stable, unique React `key` (more on why that matters below).

`useCallback` here memoizes the `submitQuestion` function itself, so child components receiving it as a prop (`ChatPanel`) don't see a "new" function identity on every unrelated re-render — relevant for optimization, less critical at this app's scale, but a pattern you'll see everywhere in React code.

`App.jsx` — wiring it together

jsx

```jsx
const { messages, activeTrace, isTraceOpen, setIsTraceOpen, isStreaming, error,
const isDesktop = useMediaQuery('(min-width: 768px)')
```

`App` calls the two hooks, gets back state + functions, and passes pieces of that down as props to `ChatPanel` and the trace panel. This top-down flow (state lives high in the tree, gets threaded down via props to whoever needs it) is called **"lifting state up"** and is the default state-management strategy for an app this size — no Redux, no Context API needed, because the tree is shallow and the prop-passing stays readable (`tracePanelProps` bundles four props into one spread to keep the JSX tidy).

`isDesktop` then drives a genuinely structural decision — which whole layout branch renders:

jsx

```jsx
{isDesktop ? (
  <Group orientation="horizontal" className="flex-1 overflow-hidden">
    <Panel defaultSize="72" minSize="45"><ChatPanel .../></Panel>
    <Separator .../>
    <Panel defaultSize="28" minSize="20" maxSize="45"><TracePanel .../></Panel>
  </Group>
) : (
  <div className="flex flex-1 flex-col overflow-hidden">
    <div className="max-h-72 shrink-0 ..."><TracePanel .../></div>
    <div className="min-w-0 flex-1"><ChatPanel .../></div>
  </div>
)}
```

Desktop gets a draggable split-pane (chat left, trace right) via the `react-resizable-panels` library (`Group`/`Panel`/`Separator`); mobile gets a stacked layout with the trace collapsed above the chat. Same `TracePanel` and `ChatPanel` components, same props, two different arrangements — this is the payoff of building UI out of small composable components.

1.4 The full component tree

```
main.jsx                          (mounts React into <div id="root"> from
index.html)
  └─ App.jsx                      (layout + wires useChat/useMediaQuery)
     ├─ header: Sprout icon, title, InfoTooltip.jsx
     ├─ DisclaimerBanner.jsx      (persistent "not medical advice" strip)
     └─ ChatPanel.jsx
        ├─ EmptyState             (pipeline-steps illustration, example
prompts)
        ├─ per-message bubbles
        │     ├─ RetryBadge.jsx
        │     └─ CitationList.jsx
        └─ input <form>
     └─ TracePanel  (defined right inside App.jsx, not its own file)
        └─ ReasoningTrace.jsx
```

Worth noting: `TracePanel` isn't in its own file — it's a small function defined at the top of `App.jsx`, because it's only ever used in two spots inside that same file (desktop and mobile branches) and has no reuse value elsewhere. Not every visually-distinct chunk needs to become a separate component file; the split usually tracks "does this need to be reused or independently testable," not "is it visually a box."

`CitationList.jsx` — dedup + the `key` prop

jsx

```jsx
const seen = new Set()
const uniqueCitations = citations.filter((citation) => {
  if (seen.has(citation.pmid)) return false
  seen.add(citation.pmid)
  return true
})
...
{uniqueCitations.map((citation) => (
  <li key={citation.pmid}> ... </li>
))}
```

Plain JS `Set`/`filter` — nothing React-specific about the dedup itself. The React-specific part is `key={citation.pmid}`: whenever you render a list with `.map()`, React needs a **stable identity** for each item across renders so it can tell "this is the same citation, just

reorder/update it" apart from "this is a brand-new one, mount it fresh." Using the PMID (a real, stable identifier) is correct; using the array *index* as the key (a common beginner mistake) breaks this the moment the list order or contents change, causing subtle re-render bugs. `ReasoningTrace.jsx` actually uses `index` as its key deliberately — acceptable there because trace events are pure, append-only, and never reordered.

`ReasoningTrace.jsx` — the most "logic-heavy" UI component

Three patterns worth pulling out:

1. **Lookup-table components.** `NODE_META` maps a backend node name (`"grade_documents"`) to a `{ label, Icon }` pair. Instead of a chain of `if/else`, the render code does `NODE_META[event.node] ?? { label: event.node, Icon: Circle }` — a single object lookup with a sane fallback. `TONE_STYLES` does the same for color styling. This is the frontend equivalent of a backend dispatch table / strategy pattern, and it's the idiomatic React way to avoid sprawling conditional JSX.
2. **Derived "retry" detection via a custom hook:**

```
js

function useRepeatFlags(events) {
  return useMemo(() => {
    const counts = {}
    return events.map((event) => {
      counts[event.node] = (counts[event.node] || 0) + 1
      return counts[event.node] > 1
    })
  }, [events])
}
```

The backend never tells the frontend "this is a retry" explicitly — the frontend *infers* it by noticing a node name (e.g. `"retrieve"`) appearing a second time in the event stream, meaning the corrective loop kicked in. All the retry-colored badges and dots you see in the UI trace back to this one counting function.

3. **CSS-only timeline line.** The vertical connecting line between trace steps isn't an SVG or a library — it's one `<span>` absolutely positioned and stretched with Tailwind utilities (`absolute left-[13px] top-7 h-[calc(100%+4px)] w-px bg-stone-200`), skipped on the last item (`!isLast && ...`). A good example of how far plain CSS positioning gets you before you need a real graphics library.

`lib/formatAnswer.jsx` — a small but instructive regex

```
js
```

```
export function renderInlineMarkdown(text) {
  const parts = text.split(/(\*\*[^*]+\*\*)/g)
  return parts.map((part, index) =>
    part.startsWith('**') && part.endsWith('**')
      ? <strong key={index}>{part.slice(2, -2)}</strong>
      : part
  )
}
```

`String.split()` with a **capturing group** `(...)` in the regex keeps the matched delimiters *in* the resulting array instead of discarding them — so splitting `"the **STAR trial** found"` gives you `["the ", "**STAR trial**", " found"]` in one pass, not just the non-bold pieces. That's why this function can return a mixed array of plain strings and `<strong>` elements — React is perfectly happy rendering an array of mixed types as children.

1.5 `lib/streamQuery.js` — the file doing the real work

This is the piece actually talking to your FastAPI backend, so it deserves the deepest look. Skim `api/main.py` again: `/query/stream` returns a `StreamingResponse` with `media_type="text/event-stream"` — that's **Server-Sent Events (SSE)**, a simple one-way (server → client) streaming protocol, much lighter-weight than WebSockets, well suited to "push updates as they happen, no need for the client to send anything back mid-stream."

The wire format is just text:

```
event: node_update
data: {"node": "retrieve", "detail": "..."}

event: done
data: {"answer": "...", "citations": [...], ...}
```

Each event is separated from the next by a **blank line**.

Browsers ship a built-in `EventSource` API for consuming SSE — but it's **GET-only**, and your endpoint needs to receive the question as a JSON POST body. So `streamQuery.js` hand-rolls SSE parsing on top of `fetch()`'s raw streaming body instead:

```
js
```

```
const response = await fetch(`${API_BASE}/query/stream`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ question }),
})

const reader = response.body.getReader()
const decoder = new TextDecoder()
let buffer = ''

while (true) {
  const { value, done } = await reader.read()
  if (done) break
  buffer += decoder.decode(value, { stream: true })

  let boundary = buffer.indexOf('\n\n')
  while (boundary !== -1) {
    const rawEvent = buffer.slice(0, boundary)
    buffer = buffer.slice(boundary + 2)
    dispatchEvent(rawEvent, { onNodeUpdate, onDone })
    boundary = buffer.indexOf('\n\n')
  }
}
```

Walk through why every piece is there:

- `response.body` is a **`ReadableStream`** — bytes arrive in unpredictable chunks as the network delivers them, not all at once. This is directly analogous to reading a socket on the backend: you get *some* bytes, maybe a partial message, and have to buffer until you have a complete unit.
- `reader.read()` pulls the next chunk; `done: true` means the server closed the stream (equivalent to hitting EOF on a socket).
- `TextDecoder` turns raw bytes into a JS string. `{ stream: true }` matters: a chunk boundary can land **in the middle of a multi-byte UTF-8 character**, and this flag tells the decoder to hold back an incomplete trailing byte sequence and prepend it to the next chunk instead of corrupting it.
- `buffer` accumulates text across chunks because one `fetch` chunk might contain zero, one, or several complete SSE events — the inner `while` loop drains every *complete* event currently sitting in the buffer (delimited by `"\n\n"`), leaving any trailing partial event for the next read.
- `dispatchEvent` then parses one raw event block's `event:`/`data:` lines and calls the matching callback (`onNodeUpdate` for `node_update`, `onDone` for `done`) with the JSON-parsed payload.

This whole file is a good illustration of a general truth about frontend work: most of the genuinely hard logic in a UI isn't about pixels, it's about **correctly handling asynchronous, partial, out-of-order data** — the same kind of problem you already solve on the backend, just facing the other direction across the wire.

`useChat.js`'s `submitQuestion` is the *consumer* of this: it passes three callbacks in, and every trace bubble / final message / error banner you see on screen originates from one of `onNodeUpdate` / `onDone` / `onError` firing.

1.6 Tailwind CSS — utility-first styling

Traditional CSS: write a stylesheet with your own class names (`.chat-bubble`), then reference them. Tailwind instead ships hundreds of tiny, single-purpose utility classes (`rounded-2xl`, `px-4`, `py-2.5`, `bg-brand-600`, `shadow-sm`) that you compose directly on the element:

```jsx
<div className="max-w-[85%] rounded-2xl rounded-br-sm bg-brand-600 px-4 py-2.5
```

No separate `.css` file for this bubble exists — the styling *is* the class list. The trade-off: markup gets visually noisy, but you never have the classic CSS problem of hunting through stylesheets to find what rule is actually winning, and you never accumulate unused, half-forgotten CSS classes — because there's no "one class does five different things depending on context" indirection.

How the classes even work: `index.css`

```css
@import "tailwindcss";

@theme {
  --font-sans: "Inter", ui-sans-serif, system-ui, sans-serif;
  --color-brand-600: #1f5f5b;
  ...
}
```

`@theme` defines **design tokens** as CSS custom properties. Tailwind's build plugin (registered in `vite.config.js` as `tailwindcss()`) reads this, and every token automatically becomes a family of utility classes — `--color-brand-600` gives you `bg-brand-600`, `text-brand-600`, `border-brand-600`, etc. for free. This is how the app gets a bespoke teal/terracotta palette instead of Tailwind's stock colors — one token definition, then it's usable everywhere by name.

Just-in-time compilation

Nobody ships "all of Tailwind" to the browser — that would be megabytes of unused CSS. The Vite plugin scans your actual `.jsx` source files at build time, collects every class name that's literally present as a string, and generates a CSS file containing *only those*. This is why the class names have to be complete, static-ish strings in your source (`bg-brand-600`, not programmatically concatenated as `"bg-" + color + "-600"`) — the scanner can't execute your JS, it just looks for recognizable patterns in the text.

Responsive design: mobile-first breakpoint prefixes

```jsx
className="px-4 py-3 sm:px-6"
```

Unprefixed utilities apply at all sizes; `sm:`, `md:`, etc. apply **from that breakpoint up**. So `px-4 py-3 sm:px-6` reads as "16px horizontal padding by default, bumped to 24px once the viewport is ≥640px." `App.jsx`'s `isDesktop` React-level check (`min-width: 768px` via `useMediaQuery`) and Tailwind's own `md:` CSS breakpoints are two *separate* mechanisms doing similar-flavored work at different layers — one swaps entire component trees in JS, the other adjusts styling of the same DOM via CSS media queries. Both exist here because swapping the split-pane vs stacked *layout* needs actual JS branching, while padding/sizing tweaks are cheap enough to leave to CSS.

Where plain CSS still shows up

```css
@keyframes fade-slide-in { from { opacity: 0; transform: translateY(6px); } to
.animate-fade-slide-in { animation: fade-slide-in 0.35s ease-out both; }
```

Custom keyframe animations (the fade-in on new trace steps, the pulsing dot on the send button while streaming) are hand-written CSS, not Tailwind utilities — Tailwind isn't a replacement for CSS, it's a fast way to write the 90% of styling that's just spacing/color/typography, and you drop to raw CSS for the remaining custom bits, then reference the result (`.animate-*`) as if it were just another utility class.

1.7 Vite — the build tool and dev server

Two completely different jobs, one tool:

Dev mode (`npm run dev`): Vite starts a local server (default port 5173) and serves your source files essentially as-is, leaning on the browser's *native* ES module support (`import`/`export`) rather than pre-bundling everything upfront. That's why it starts almost instantly even on a large app — compare to older tools (Webpack) that had to bundle the entire dependency graph before serving a single page. When you save a file, Vite pushes just that module to the browser and React's Fast Refresh patches the running app **without**

losing component state — edit `RetryBadge.jsx` while a chat is mid-conversation, and the conversation doesn't reset.

Build mode (`npm run build`): for production, native unbundled ES modules would mean hundreds of separate network requests per page load — bad for real users on real networks. So `build` switches to actually bundling, minifying, and tree-shaking (dropping unused code) everything into a small number of static files under `frontend/dist/`. Those output files are pure HTML/CSS/JS with zero runtime dependency on Node.js or Vite — which is exactly why they can be handed to Vercel (a static host) and just... work, no server process required. This is the fundamental reason the frontend needs no Docker container: there's nothing left to "run" after the build step, only files to serve.

```js
// frontend/vite.config.js
export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

Two plugins: `react()` handles the JSX→JS transform plus Fast Refresh; `tailwindcss()` handles scanning + generating the CSS described above.

Env vars: `import.meta.env.VITE_API_BASE`

```js
const API_BASE = import.meta.env.VITE_API_BASE ?? 'http://127.0.0.1:8000'
```

Vite only exposes environment variables to client-side code if they're prefixed `VITE_` (see `frontend/.env.example`: `VITE_API_BASE=...`). This isn't a style convention, it's a **security boundary**: anything without that prefix is deliberately kept server/build-side only, so you can't accidentally reference a secret (say, a `DATABASE_URL`) in frontend code and have it get bundled — in plaintext, readable by anyone — into the JS file shipped to every visitor's browser. Contrast with your `OPENAI_API_KEY` in the *backend's* `.env` — that one is safe precisely because it never goes near a bundler that ships code to end users.

1.8 npm & `package.json` — the dependency toolchain

```json
```

```
"dependencies": {
  "@tailwindcss/vite": "...", "lucide-react": "...", "react": "...",
  "react-dom": "...", "react-resizable-panels": "...", "tailwindcss": "..."
},
"devDependencies": {
  "@types/react": "...", "@types/react-dom": "...",
  "@vitejs/plugin-react": "...", "oxlint": "...", "vite": "..."
}
```

`dependencies` vs `devDependencies` is a real, meaningful split (unlike a flat `requirements.txt`): `dependencies` are things your *code imports and runs in the browser* (React itself, the icon library, the resizable-panel library). `devDependencies` are tools only needed on your machine while developing/building (Vite, the React plugin, the linter, TypeScript type definitions for editor autocomplete) — none of this ships to the browser. `npm run build` uses devDependencies to *produce* the output but none of their code ends up in `dist/`.

- `npm install` reads `package.json`, resolves the full dependency tree, and writes `package-lock.json` — the exact-versions-pinned equivalent of a fully resolved `poetry.lock`/pip-compiled requirements file, ensuring everyone (and CI, and the Docker build if there were one) installs byte-identical dependency versions.
- Packages get installed into `node_modules/` — this repo's rough equivalent of a virtualenv, except there's no "activate" step; tools just resolve imports from `node_modules` automatically, and it's *always* per-project (no global site-packages-style default), which is why it's gitignored and regenerated by `npm install` on a fresh clone.
- `"scripts"` are named shortcuts run via `npm run <name>` — `dev` (Vite dev server), `build` (production bundle), `lint` (runs `oxlint`, a fast Rust- based linter), `preview` (locally serve the built `dist/` to sanity-check a production build before deploying).

1.9 The full journey of one question, end to end

Putting it all together — what actually happens when a user types a question:

1. User types into the `<input>` in `ChatPanel`; each keystroke fires `onChange` → `setInput(event.target.value)` → re-render with the new text visible (controlled input).
2. Submit → `ChatPanel`'s `handleSubmit` calls `onSubmit(input)`, which is the `submitQuestion` function `App` received from `useChat()` and passed down as a prop.
3. `submitQuestion` resets trace/error state, sets `isStreaming = true` (which re-renders the send button as a pulsing dot and disables the input), and calls `streamQuery(...)`.
4. `streamQuery` opens a `fetch()` POST to your Dockerized FastAPI backend's `/query/stream`, and starts reading the SSE byte stream as it arrives.

5. Each `node_update` SSE event → `onNodeUpdate` → pushed into the local `trace` array → `setActiveTrace([...trace])` → `ReasoningTrace` re-renders with one more timeline step, live, mid-request.
6. The final `done` SSE event → `onDone` → a new message object appended to `messages` state → `ChatPanel` re-renders with the new question/answer bubble, `RetryBadge`, and deduped `CitationList`; `isStreaming` flips back to `false`.
7. Any network/parsing failure along the way → `onError` → `error` state set → the red error banner under `ReasoningTrace` appears.

Every visual behavior in this app is explainable as "some piece of state changed, so the relevant component function re-ran." That's the whole mental model of React, applied concretely.

PART 2 — Docker, from first principles

2.1 The problem Docker solves

"It works on my machine" happens because your machine has a particular OS, Python version, system libraries, and installed packages — and the deploy target might not match any of that. Docker's answer: package the **entire runtime environment** (OS layer, system deps, language runtime, your code) into one artifact — an **image** — that runs identically anywhere Docker itself runs.

- An **image** is a read-only snapshot/template — think of it like a class definition, or a compiled build artifact.
- A **container** is a running (or stopped) *instance* of an image — think of it like an object instantiated from that class. You can start many containers from one image.
- Unlike a full virtual machine, a container doesn't virtualize an entire OS — it shares the host machine's kernel and just gets an isolated filesystem/process/network view. That's why containers start in milliseconds and images are megabytes, not gigabytes, compared to VMs.

2.2 `Dockerfile`, line by line

```dockerfile
FROM python:3.11-slim
```

Every image builds *on top of* a base image. `python:3.11-slim` is an official image: a minimal Debian Linux with Python 3.11 preinstalled. "slim" trades a smaller image size for omitting some build tools/headers you might need for packages with C extensions — a deliberate size/completeness trade-off, not a mistake.

```dockerfile
WORKDIR /app
```

Sets the working directory *inside the image's filesystem* for every instruction that follows — like `cd /app` inside the container, permanently, for both build-time `COPY`/`RUN` steps and the eventual runtime `CMD`.

dockerfile

```
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

Here's the single most important Docker concept to internalize: **each instruction becomes a cached layer.** Docker builds an image as a stack of layers, and if the *inputs* to a given instruction haven't changed since the last build (same base image, same preceding layers, same files being copied in), Docker reuses the cached result instead of re-executing it.

This is exactly why `requirements.txt` is copied and installed **before** the application code — the comment in your own Dockerfile says so directly. Your source code (`agent/`, `api/`) changes on nearly every commit; `requirements.txt` changes rarely. If dependency installation happened *after* copying source code, every single code change would invalidate the cache for the `pip install` layer too, forcing a full dependency reinstall on every build — turning a 2-second rebuild into a multi-minute one. Ordering instructions from "changes least often" to "changes most often" is the single highest-leverage Dockerfile optimization, and this file already does it correctly.

dockerfile

```
COPY agent/ agent/
COPY api/ api/
COPY data_pipeline/ data_pipeline/
```

Selective copying, not `COPY . .` — only what `api/main.py` actually imports at runtime goes into the image. `eval/`, `frontend/`, test files, and raw data never enter the image at all, keeping it smaller and avoiding accidentally shipping things like notebooks or local artifacts.

dockerfile

```
RUN useradd --create-home --uid 1000 appuser && chown -R appuser:appuser /app
USER appuser
```

By default, a container's main process runs as **root** — inside the container's own isolated namespace, so it's not literally root on your host machine, but it's still an unnecessary privilege if the process is compromised (e.g. via a dependency vulnerability, an attacker who gets code execution inside the container has full control of everything in it). Creating a dedicated unprivileged `appuser`, handing it ownership of `/app`, and switching to it with

`USER appuser` before the app runs is a standard hardening step — the app genuinely doesn't need root to serve HTTP requests on port 8000.

dockerfile

```
EXPOSE 8000
```

This is **documentation only** — a note to humans and tooling about which port the containerized app listens on. It does **not** actually make the port reachable from outside the container; that's `docker-compose.yml`'s `ports:` section, below. A very common early misconception is thinking `EXPOSE` alone opens the port — it doesn't.

dockerfile

```
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

The default command run when a container starts from this image (can be overridden at `docker run`/compose time, but usually isn't here). `--host 0.0.0.0` matters: it tells uvicorn to listen on all network interfaces *inside the container*, not just `127.0.0.1` — if it only bound to localhost, the port-mapping from outside the container wouldn't be able to reach it at all, since "localhost" inside a container refers to the container itself, not your host machine.

2.3 `.dockerignore`

```
.git / .venv/ / __pycache__/          → not needed in the image at all
.env                                  → secrets — injected at runtime instead,
see below
data/ / chroma_db/                    → mounted via a volume at runtime, not
baked in
frontend/node_modules/, dist/, .env → the frontend is a separate deploy
target entirely
```

Functionally identical concept to `.gitignore`, but it controls the **build context** — the set of files Docker is even allowed to `COPY` from during a build. Excluding `.env` here is a real security decision, not tidiness: anything baked into an image *layer* is recoverable by anyone who gets the image, even if a later layer "deletes" it — image layers are individually inspectable and layers are cumulative. Instead, secrets are injected as plain environment variables into the *running container's process* at startup, via `docker-compose.yml`'s `env_file: .env` — so the secret exists only in memory of a live container, never as a byte in any image layer that might get pushed to a registry or shared.

2.4 `docker-compose.yml`, line by line

yaml

```yaml
services:
  backend:
    build: .
```

`docker compose` orchestrates one or more **services**, each backed by a container. `build: .` means "build the image from the `Dockerfile` in this directory" (as opposed to `image: someuser/prebuilt-image`, which would pull an already-built image from a registry instead of building locally). There's only one service here — no `frontend` entry — for exactly the reason from Part 0: the frontend has nothing that needs to *run* as a process.

```yaml
    container_name: corrective-rag-backend
```

A human-readable name instead of Docker's default randomly-generated one (`recursing_edison`-style) — purely for convenience when running `docker ps`, `docker logs`, etc.

```yaml
    ports:
      - "8000:8000"
```

`HOST:CONTAINER` port mapping. A container has its own isolated network namespace by default — nothing on your host machine can reach anything inside it unless explicitly published. This line says "forward my host machine's port 8000 to this container's port 8000," which is what makes `http://localhost:8000/health` reachable from your browser or `curl` at all.

```yaml
    env_file:
      - .env
```

Loads every `KEY=value` line from `.env` (your `OPENAI_API_KEY`, `NCBI_API_KEY`) as environment variables inside the container's process — this is the runtime injection mechanism referenced above, and it's why `api/main.py` can call `load_dotenv()`/read `os.environ` normally without the key ever having been part of the image build.

```yaml
    volumes:
      - ./chroma_db:/app/chroma_db
```

A **bind mount**: it literally maps a folder on your *host* filesystem (`./chroma_db`, the vector index you build locally via `python -m data_pipeline.build_index`) to a path *inside* the container (`/app/chroma_db`). Two consequences:

- The container sees, at startup, whatever is currently on your host in that folder — no need to rebuild the image every time you regenerate the index.
- Any writes the container makes to that path flow back out to your host folder too, and — critically — **that data survives the container being removed and recreated.** Anything written *inside* a container but not in a mounted volume disappears the instant the container is deleted; the container's writable layer is itself temporary. Volumes/bind-mounts are the escape hatch for anything that needs to persist.

(The alternative would be a **named volume** — storage Docker manages itself in its own location, rather than a path you choose on the host. The comment in your compose file explains the bind-mount choice explicitly: it lets you reuse the index you already built locally, directly, without any extra copy step.)

yaml

```yaml
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlo
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 10s
```

A command Docker runs **inside the running container**, on a schedule, to decide whether the container is actually healthy — not just "the process hasn't crashed," but "it's genuinely serving traffic." It literally calls your own `GET /health` endpoint from `api/main.py` using Python's stdlib (no extra dependency needed just for this check). Reading the fields: check every `interval` (30s); if a single check takes longer than `timeout` (5s), it counts as a failure; after `retries` (3) consecutive failures the container is marked `unhealthy`; `start_period` (10s) is grace time after startup before failures start counting, so the container isn't marked unhealthy just because uvicorn hadn't finished booting yet. This status is what tools like `docker ps` or an orchestrator would use to decide whether to route traffic to this container or restart it.

yaml

```yaml
    restart: unless-stopped
```

A **restart policy** — if the container's process crashes, or the host reboots, Docker automatically starts it again, *unless* you explicitly ran `docker stop` on it yourself (in which case it stays stopped until you start it again manually). This is what makes a Docker-run backend behave like a proper always-on service rather than something you have to babysit.

2.5 Actually using it

```bash
docker compose build      # build (or rebuild) the image from the Dockerfile
docker compose up -d      # create + start the container, detached (-d = backgr
docker compose logs -f backend   # follow the running container's stdout/stderr
docker compose down       # stop and remove the container (image stays cached)
```

`docker-compose.yml` exists so that "build this image, run it with this port mapping, these env vars, this volume, this health check, this restart policy" is one declarative file and one `up` command — instead of a single `docker run` command with a dozen flags that's easy to get subtly wrong between runs.

2.6 Frontend + Dockerized backend, talking to each other

Two loose ends tie directly back to Part 1:

CORS. `api/main.py`:

```python
DEV_FRONTEND_ORIGINS = ["http://localhost:5173", "http://127.0.0.1:5173"]
app.add_middleware(CORSMiddleware, allow_origins=DEV_FRONTEND_ORIGINS, ...)
```

Browsers enforce the **Same-Origin Policy**: JavaScript running on one origin (`http://localhost:5173`, your Vite dev server) is blocked by the *browser itself* from reading responses from a different origin (`http://localhost:8000`, your Dockerized backend) unless that server explicitly opts in via CORS headers. This restriction is purely browser-side — it's exactly why `curl http://localhost:8000/query` from your terminal works fine with no CORS setup at all, but the same request made by `fetch()` from a page served on port 5173 would be silently blocked without this middleware. It has nothing to do with Docker or ports being "closed" — it's the browser refusing to hand JS the response.

The URL that closes the loop. `frontend/.env` (`VITE_API_BASE`) points at `http://127.0.0.1:8000` — the host-side port that `docker-compose.yml`'s `ports: "8000:8000"` publishes. `streamQuery.js` uses exactly that value to build its `fetch()` URL. Change the compose port mapping, and this env var is the one thing on the frontend side that needs to change to match.

Mental model, compressed

- **Frontend = a pure function of state, running in the user's browser.** React re-renders whenever state changes; Tailwind styles it via composed utility classes; Vite compiles/serves it; the end product of `npm run build` is static files needing no server at all.

- **Docker = "ship the whole runtime environment, not just the code."** The Dockerfile builds one reusable image (layered, cached, least-to-most volatile ordering); docker-compose runs it as a container with the networking, secrets, persistence, and health-checking wired up declaratively.
- **The two halves only ever meet over plain HTTP** — a `fetch()` call from a browser to a port your compose file published — which is exactly why they can be developed, deployed, and reasoned about almost entirely independently.

Good next step if you want this to *stick* rather than just be read once: run `docker compose up -d`, open your browser's DevTools Network tab, submit a question, and watch the actual `/query/stream` request — you'll see the SSE events arrive one at a time in real time, which is the clearest way to watch Part 1.5 and Part 2 actually meet.